

Method Overriding in C# with Examples

In this article, I am going to discuss **Method Overriding in C#** with Examples. Please read our previous article where we discussed [Operator Overloading in C#](#) with Examples. Here in this article, we are going to discuss the following pointers with examples.

1. **What is Method Overriding in C#?**
2. **When do we need to override a method in C#?**
3. **When a subclass method is treated as an overridden method in C#?**
4. **How a method is Overridden in C#?**
5. **Multiple Examples to Understand Method Overriding in C#?**
6. **How to execute the superclass method if it is overridden in the subclass?**
7. **Method Overriding Realtime Example in C#.**
8. **What are the differences between Method Overloading and Method Overriding in C#?**

Note: The terms Function Overriding and Method Overriding are interchangeably used. Method Overriding is an approach to implementing Polymorphism (i.e. Run-Time Polymorphism or Dynamic Polymorphism) in C#.

What is Method Overriding in C#?

The process of re-implementing the superclass non-static, non-private, and non-sealed method in the subclass with the same signature is called Method Overriding in C#. The same signature means the name and the parameters (type, number, and order of the parameters) should be the same.

When do we need to override a method in C#?

If the Super Class or Parent Class method logic is not fulfilling the Sub Class or Child Class business requirements, then the Sub Class or Child Class needs to override the superclass method with the required business logic. Usually, in most real-time applications, the Parent Class methods are implemented with generic logic which is common for all the next-level sub-classes.

When is a Sub Class Method treated as an Overriding Method in C#?

If a method in the sub-class or child class contains the same signature as the superclass non-private, non-static, and non-sealed method, then the subclass method is treated as the overriding method and the superclass method is treated as the overridden method.

How can we Override a Parent Class Method under Child Class in C#?

If you want to override the Parent class method in its Child classes, first the method in the parent class must be declared as virtual by using the virtual keyword, then only the child classes get permission for overriding that method. Declaring the method as virtual is marking the method as overridable. If the child class wants to override the parent class virtual method, then the child class can override it with the help of the override modifier. But overriding the parent class virtual methods under the child classes is not mandatory. The syntax is shown below to implement Method Overriding in C#.

```

class Class1
{
    //Virtual Function (Overridable Method)
    public virtual void Show()
    {
        //Parent Class Logic Same for All Child Classes
    }
}

class Class2 : Class1
{
    //Overriding Virtual Method
    public override void Show()
    {
        //Child Class Reimplementing the Logic
    }
}

class Class3 : Class1
{
    //Overriding Virtual Method is Optional
}

```

As you can see in the above image, the Show method is declared as a Virtual method in the class Class1. Further Class1 is the Parent class for Class2 and Class3. Class2 overrides the Show method while class Class3 does not override the Show method as overriding the virtual method inside a child class is optional.

Suppose, on your birthday, your parents give you a mobile phone. Then your parents tell you, if you like it, it's OK, you just use it. And if you don't like the mobile phone, then you can exchange it. So, if you want to exchange it, take the bill, go and exchange the mobile phone. So, you have two options. What are these? The first option, whatever your parent gives you, you just use it. The second option, if you don't like it, go and exchange and take whatever you like.

This is exactly the same as Method overriding. You have one method in the Parent class, and that method is given to the child class for consumption. Now, if the child class wants then the child class can consume the method else the child class can reimplement the method or override the method. By declaring the Parent method as virtual, it gives permission to the child classes to override the method and the child classes can override the method by using the override modifier.

Example to Understand Method Overriding in C#

Let us see an example for understanding Method Overriding in C#. Please have a look at the below code. Here class Class1 is the parent class and in this class, we defined one method i.e. Show() by using the virtual keyword which marks this method to be overridable inside the child classes. The class Class2 is derived from the class Class1 and hence it becomes a child class of class Class1 and as soon as it becomes a child class, it got permission to override the overridable method Show(). As you can see in the child class, we override the Show method by using the override modifier.

```

using System;
namespace PolymorphismDemo
{

```

```

class Class1
{
    //Virtual Function (Overridable Method)
    public virtual void Show()
    {
        //Parent Class Logic Same for All Child Classes
        Console.WriteLine("Parent Class Show Method");
    }
}
class Class2 : Class1
{
    //Overriding Method
    public override void Show()
    {
        //Child Class Reimplementing the Logic
        Console.WriteLine("Child Class Show Method");
    }
}

class Program
{
    static void Main(string[] args)
    {
        Class1 obj1 = new Class2();
        obj1.Show();
        Class2 obj2 = new Class2();
        obj2.Show();
        Console.ReadKey();
    }
}

```

Output:

```

Child Class Show Method
Child Class Show Method

```

Why in both cases the Child Class Show Method is Invoked?

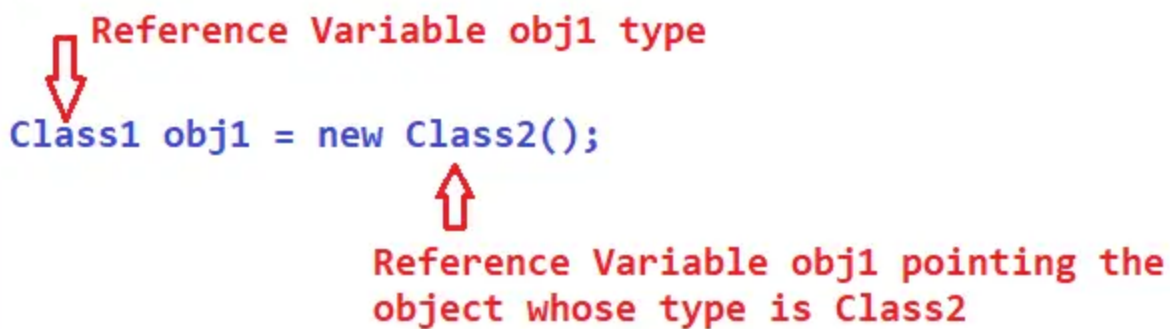
While working with Polymorphism in C# we need to understand two things i.e. what happens at the time of compilation and what happens at the time of execution for a method call. Is the method going to be executed from the same class at run-time which is bounded to the class at the compile time or is the method going to be executed from a different class at run-time rather than the class bounded at compile time? Let us understand this.

In our example, we have written the following code inside the Main method.

```
static void Main(string[] args)
{
    Class1 obj1 = new Class2();
    obj1.Show();

    Class2 obj2 = new Class2();
    obj2.Show();
    Console.ReadKey();
}
```

Now, let us understand what Compiler and CLR do when each statement is executed. Please observe the first statement. Here, you can see that the reference variable obj1 type is Class1 and this obj1 reference variable points to the object whose type is Class2.



Reference Variable obj1 type

Reference Variable obj1 pointing the object whose type is Class2

Here, Class1 is the superclass and Class2 is the subclass. The point that you need to keep in mind is that the Super Class Reference Variable Can hold the Subclass object reference and but the reverse is not possible i.e. Sub Class Reference Variable can never hold the Super Class Object Reference. So, the Class1 reference variable can hold the Class2 object reference.

Now, observe the following statement. Here, the obj1 reference variable type is Class1 and obj1 points to the object whose type is Class2. Then using obj1 we are invoking the Show() method. Now, let us try to understand what happens at compilation time and at runtime for the following method call.

obj1.Show();

obj1 => Reference Variable Type= Class1
obj1 => Pointing to Object whose type = Class2

Compilation => Compiler will check the Reference Variable Type
Runtime => CLR will check the Object Type

At the time of compilation, the function call will bind with its function definition. That means the compiler will create a link between the function call and function definition. To bind the function, the compiler will check the type of the reference variable and will check if that method or function is available in that type or not. If available then compilation will be done successfully and if not available then compilation will fail and you will get a compilation error. In our example, the Show method is available in Class1 (type of the reference variable obj1) and hence compilation is successful.

At the time of program execution, the CLR will check the object type and it is going to execute the method from the reference object type. If the method is not available in the corresponding object type, then it will try to execute the method from the Parent class of the object type. In our case, the Show method is available in the class Class2 and hence this method is going to be executed from the class Class2. This is because of method overriding and this is also called Dynamic Polymorphism or Runtime Polymorphism.

What is Dynamic Polymorphism or Runtime Polymorphism?

The function call is bounded to the class at the time of compilation, if the function is going to be executed by CLR from a different class at run-time rather than the class bounded at compilation-time, then it is called Run-Time Polymorphism in C#. This happens in the case of Method Overriding because, in the case of Overriding, we have multiple methods with the same signature i.e. Parent Class and the Child class have the same method implementation. So, in this case, we will be able to know at runtime from which class the method is going to be executed.

It is also called Dynamic Polymorphism or Late Binding as at Run-time we will be able to know from which class the method is going to be executed.

What is Static Polymorphism or Compile Time Polymorphism?

The function call is bounded to the class at the time of compilation, if the function is going to be executed from the same bounded class at run-time, then it is called Compile-Time Polymorphism in C#. This happens in the case of Method Overloading because, in the case of overloading each method will have a different signature and based on the method call, we can easily recognize the method which matches the method signature.

It is also called Static Polymorphism or Early Binding as at the Compilation time we will be able to know from which class the method going to be executed.

Now, please observe the following code. Here, the reference variable obj2 type is Class2 and it is also pointing to the object reference whose type is Class2. Then using the obj2 reference variable we are invoking the Show method.

```
Class2 obj2 = new Class2();  
obj2.Show();
```

At the time of compilation, the compiler will check whether the Show method is available inside the Class2 reference variable and it will find that method is available and hence compilation is successful. Then at runtime, the CLR will check the method definition inside the object type i.e. Class2 and it finds that method is available inside the Class2 and it will execute that method from the Class2. So, in both, the method call the method is going to be executed from the child class as both the reference variable are pointing to the child class object.

Note: The point that you need to keep in mind is that the overriding method is always going to be executed from the current class object. The superclass method is called the overridden method and the sub-class method is called the overriding method.

Overriding the Virtual Method is Optional in C#:

The point that you need to keep in mind is overriding the virtual method in the child classes is optional. If you are not overriding the virtual method means you going with the default implementation which is provided by the superclass. Let us understand this with an example. In the below example, inside the Parent class Class1, we marked the Show method as virtual but inside the child class Class2, we have not overridden the method. In this case, always the method is going to be executed from the Parent class only.

```
using System;
namespace PolymorphismDemo
{
    class Class1
    {
        //Virtual Function (Overridable Method)
        public virtual void Show()
        {
            //Parent Class Logic Same for All Child Classes
            Console.WriteLine("Parent Class Show Method");
        }
    }
    class Class3 : Class1
    {
        //Not Overriding the Virtual Method
    }

    class Program
    {
        static void Main(string[] args)
        {
            Class3 obj3 = new Class3();
            obj3.Show();
            Class1 obj4 = new Class3();
            obj4.Show();
            Console.ReadKey();
        }
    }
}
```

Output:

```
Parent Class Show Method
Parent Class Show Method
```

Now, let us understand the Main method code. Please observe the following code first. In this case, the type of the reference variable and the object the variable points to are the same i.e. Class3.

```
Class3 obj3 = new Class3();  
obj3.Show();
```

So, at the time of compilation, the compiler will check the Show method inside Class3 and it will not find the Show method inside this class. So, it will again go and check the Superclass of Class3 which is Class1 and it finds the method inside the Class1 and it will link that method definition from Class1 with the method call.

At the time of execution, the CLR will try to execute the method from the object type which is Class3 in this case and it will not find the method definition inside the class Class3. So, it again goes and tries to execute the method from its superclass i.e. Class1 and it finds that the method definition is there and it will execute that method definition.

Now, observe the next function call statements as shown in the below image. In this case, the type of the reference variable is Class1 and the reference variable obj4 points to the object whose type is Class3.

```
Class1 obj4 = new Class3();  
obj4.Show();
```

At the time of compilation, the compiler will check the Show method inside the Class1 and it will find the Show method inside this class and it will link that method definition from Class1 with the method call.

At the time of execution, the CLR will try to execute the method from the object type which is Class3 in this case and it will not find the method definition inside the class Class3. So, it will go and try to execute the method from its superclass i.e. Class1 and it finds that the method definition is there and it will execute that method definition. So, in this example, for both the method call, the method is going to be executed from the Parent class.

How can we execute the superclass method if it is overridden in the sub-class in C#?

Once we re-implement the parent class methods under the child classes, then the object of the child class calls its own method but not its parent class method. But if you want to still consume or call the parent class's methods from the child class, then it can be done in two different ways.

By creating the parent class object under the child class, we can call the parent class methods from the child class, or by using the base keyword, we can call parent class methods from the child class, but this and the base keyword cannot be used under the static block.

Using the base keyword to call the Parent Class Methods in C#:

Let us see an example for a better understanding. As you can see in the below code, from the child class Show method we are calling the parent class Show method by using base.Show() method call.

```
using System;  
namespace PolymorphismDemo
```

```

{
    class Class1
    {
        //Virtual Function (Overridable Method)
        public virtual void Show()
        {
            //Parent Class Logic Same for All Child Classes
            Console.WriteLine("Parent Class Show Method");
        }
    }
    class Class2 : Class1
    {
        //Overriding Method
        public override void Show()
        {
            base.Show(); //Calling Parent Class Show method
            Console.WriteLine("Child Class Show Method");
        }
    }
    class Program
    {
        static void Main(string[] args)
        {
            Class1 obj1 = new Class2();
            obj1.Show();
            Class2 obj2 = new Class2();
            obj2.Show();
            Console.ReadKey();
        }
    }
}

```

Output:

```

Parent Class Show Method
Child Class Show Method
Parent Class Show Method
Child Class Show Method

```

Calling Parent Class Methods by creating the Parent Class Object under the Child Class:

Let us see an example for a better understanding of how to create the parent class object and call the parent class methods from the child class method. As you can see in the below example, from the child class Show method, we are creating an instance of the Parent class and calling the Parent Class Show method.

```

using System;
namespace PolymorphismDemo
{

```

```

class Class1
{
    public virtual void Show()
    {
        Console.WriteLine("Parent Class Show Method");
    }
}
class Class2 : Class1
{
    public override void Show()
    {
        //Creating an instance of Parent Class
        Class1 class1 = new Class1();
        //Calling Parent Class Show method
        class1.Show();
        Console.WriteLine("Child Class Show Method");
    }
}
class Program
{
    static void Main(string[] args)
    {
        Class1 obj1 = new Class2();
        obj1.Show();
        Class2 obj2 = new Class2();
        obj2.Show();
        Console.ReadKey();
    }
}

```

Output:

```

Parent Class Show Method
Child Class Show Method
Parent Class Show Method
Child Class Show Method

```

Method Overriding Real-Time Example in C#

We need to develop an application to calculate bonuses based on the designation of the Employees. The management team has decided to give 50000 as a fixed bonus or based only salary they may get 20% or 25% as bonus whichever is higher. Here, we are taking the example of three different designations, but you can take as many as per your requirement.

1. If the designation is Developer, then the employee gets either 50000 or 20% of the Salary as a bonus (whichever is higher).
2. If the designation is Manager, then the employee gets either 50000 or 25% of the Salary as a bonus (whichever is higher).

3. If the designation is Admin, then the employee will get a fixed 50000 as a bonus.

The following example code does the same as per our requirement.

```
using System;
namespace MethodOverriding
{
    public class Employee
    {
        public int Id { get; set; }
        public string Name { get; set; }
        public string Designation { get; set; }
        public double Salary { get; set; }
        public virtual double CalculateBonus(double Salary)
        {
            return 50000;
        }
    }
    public class Developer : Employee
    {
        //50000 or 20% Bonus to Developers which is greater
        public override double CalculateBonus(double Salary)
        {
            double baseSalary = base.CalculateBonus(Salary);
            double calculatedSalary = Salary * .20;
            if (baseSalary >= calculatedSalary)
            {
                return baseSalary;
            }

            else
            {
                return calculatedSalary;
            }
        }
    }
    public class Manager : Employee
    {
        //50000 or 25% Bonus to Developers which is greater
        public override double CalculateBonus(double Salary)
        {
            double baseSalary = base.CalculateBonus(Salary);
            double calculatedSalary = Salary * .25;
            if (baseSalary >= calculatedSalary)
            {
                return baseSalary;
            }
            else
            {
                return calculatedSalary;
            }
        }
    }
}
```

```

    }
}
}
public class Admin : Employee
{
    //return fixed bonus 50000
    //no need to override the method
}
class Program
{
    static void Main(string[] args)
    {
        Employee emp1 = new Developer
        {
            Id = 1001,
            Name = "Ramesh",
            Salary = 500000,
            Designation = "Developer"
        };
        double bonus = emp1.CalculateBonus(emp1.Salary);
        Console.WriteLine($"Name: {emp1.Name}, Designation: {emp1.Designat
Salary: {emp1.Salary}, Bonus:{bonus}");
        Console.WriteLine();
        Employee emp2 = new Manager
        {
            Id = 1002,
            Name = "Sachin",
            Salary = 800000,
            Designation = "Manager"
        };
        bonus = emp2.CalculateBonus(emp2.Salary);
        Console.WriteLine($"Name: {emp2.Name}, Designation: {emp2.Designat
Salary: {emp2.Salary}, Bonus:{bonus}");
        Console.WriteLine();
        Employee emp3 = new Admin
        {
            Id = 1003,
            Name = "Rajib",
            Salary = 300000,
            Designation = "Admin"
        };
        bonus = emp3.CalculateBonus(emp3.Salary);
        Console.WriteLine($"Name: {emp3.Name}, Designation: {emp3.Designat
Salary: {emp3.Salary}, Bonus:{bonus}");
        Console.WriteLine();
        Employee emp4 = new Developer
        {
            Id = 1004,
            Name = "Priyanka",
            Salary = 200000,
            Designation = "Developer"
        }
    }
}

```

```

    };
    bonus = emp1.CalculateBonus(emp4.Salary);
    Console.WriteLine($"Name: {emp4.Name}, Designation: {emp4.Designation},
Salary: {emp4.Salary}, Bonus:{bonus}");

    Console.Read();
}
}
}

```

Output:

```

Name: Ramesh, Designation: Developer, Salary: 500000, Bonus:100000
Name: Sachin, Designation: Manager, Salary: 800000, Bonus:200000
Name: Rajib, Designation: Admin, Salary: 300000, Bonus:50000
Name: Priyanka, Designation: Developer, Salary: 200000, Bonus:50000

```

What is the difference between Method Overloading and Method Overriding in C#?

Method Overloading in C#

1. It is an approach to defining multiple methods with the same name but with a different signature means by changing the number, type, and order of parameters.
2. Overloading a method can be performed within a class as well as between the Parent-Child classes also.
3. To overload a parent class method under the child classes, the child class does not require to take any permission from the parent.
4. This is all about defining multiple behaviors to a method.
5. Used to implement static polymorphism.
6. No separate keywords are used to implement function overloading.

Method Overriding in C#

1. It is an approach to defining multiple methods with the same name and with the same signature means the same number, type, and order of parameters.
2. Overriding of methods is not possible within the same class it must be performed under the child classes only.
3. To override a parent class method under the child classes, first, the child class requires to take permission from its parent.
4. This is all about changing the behavior of a method.
5. Used to implement dynamic polymorphism.
6. Use the virtual keyword for the base class function and override keyword in the derived class function to implement function overriding.

In the next article, I am going to discuss [Method Hiding in C#](#) with Examples. Here, in this article, I try to explain **Method Overriding in C#** with Examples. I hope this Method Overriding in C# with Examples article will help you with your need. I would like to have your feedback. Please post your feedback, question, or comments about this Method Overriding in C# with Examples article.